Analyst workspace: system design, components and security

Technical design and security reference for the analyst workspace in this repository. Unbranded and factual: product names appear only where the code depends on them or a standard is cited.

Repository	`jusso-dev/crosswire`, branch `main`
Verified at commit	`95c7f97` (merge of PR #103), 8 November 2026 (Australia/Sydney)
Scope	Web application, background worker, data model, integrations, deployment, operations and security controls as they exist at that commit, plus explicitly labelled plans and gaps
Audience	Engineers, security reviewers and operators
Source of truth	The code. Where this document and the code differ, the code wins and this document has a defect

> **How to read status labels.** Every capability carries one label:
>
> IMPLEMENTED present in code at the commit above and covered by tests or verified in the lab deployment. PARTIAL present but with a stated limitation. PLANNED tracked as an issue or explicit follow-up; not in code. GAP missing; no issue yet, or a known risk. ASSUMPTION relied on but not verified by this repository. OPEN DECISION needs an owner decision.

> **Provenance and scope check.** An earlier request described a "cohesive analyst workspace redesign". That work is real and merged: PR #98 (merge commit `0813e12`) delivered the workspace, design tokens, accessible components, URL-held view state, an automated WCAG 2.2 AA scan and Azure OpenAI chat; its pattern research and design decisions are recorded in `design.md` at the repository root. This document builds on that record rather than restating the redesign as new. Earlier history (v1 phases 0 to 8, PR #87; v2 response depth, PR #96, issues #88 to #94) is treated as the existing base. Nothing in this document describes unmerged work as implemented.

Contents

1. Summary
2. Users, workflows and boundaries
3. Architecture and trust boundaries
4. Data flows
5. Backend components
6. Data model and storage
7. Frontend: views and shared UI
8. Component and state catalogue
9. Design tokens and accessibility
10. Roles, tenants and permissions
11. Threat model
12. Security controls in depth
13. AI agent safety
14. Configuration and dependencies
15. Deployment and operations
16. Testing and acceptance criteria
17. Standards mapping
18. Gaps, open decisions and backlog
19. Traceability and references
20. Rendering this document

1. Summary

The application is a self-hosted security operations workspace for teams that use Microsoft Defender XDR and Microsoft Sentinel. Analysts keep a watch list of vendors, products, CVEs and tags; a worker matches it hourly against two public threat-intelligence feeds; analysts open hunt cases, review and approve read-only KQL searches, inspect stored result samples, and propose containment actions that an approver dry-runs and decides. A set of AI agents can read intelligence, hunts and memory and can *propose* searches or actions. They cannot run or approve anything.

Three design rules shape almost every component:

1. **People approve; machines propose.** No code path lets an agent execute KQL or a response action. Live execution is off by default and needs two independent switches.
2. **Every query is tenant-scoped on the server.** All organisation data is filtered by `org_id` in server code; UI hiding is never the control.
3. **Untrusted input stays data.** Feed content, model output, query results and webhook targets are validated, escaped or allowlisted before use.

2. Users, workflows and boundaries

2.1 Roles

Role	Purpose	Source
Analyst	Investigates, writes notes, proposes searches and actions. Default for new members.	`apps/web/src/server/response/rbac.ts`
Approver	Analyst rights plus approve, reject, dry-run and execute response proposals; schedule repeat searches.	same
Owner	Approver rights plus member roles, connectors, notification channels, audit log and audit bundle export.	same, and `fns/settings.ts`, `fns/approvals.ts`

2.2 Implemented workflows

#	Workflow	Status	Primary code
W1	Watch: add vendors, keywords, CVEs, tags; hourly matching against pwn.watch and CISA KEV; daily digest per `APP_TIMEZONE` day	IMPLEMENTED	`server/ti/*`, `routes/app.watches.tsx`, `routes/app.digests.tsx`
W2	Hunt: case file with hypothesis, KQL proposals, approval, stored sample results, evidence timeline, notes, export	IMPLEMENTED	`server/hunting/*`, `routes/app.hunts.$huntId.tsx`
W3	Repeat search: approver schedules a search that already ran; approval lapses after at most 30 days	IMPLEMENTED	`server/hunting/schedules.ts`
W4	Respond: proposal, dry run, separate approver decision, optional live execution, rollback as a new proposal	IMPLEMENTED (live execution off by default)	`server/response/*`, `routes/app.approvals*.tsx`
W5	Compare one CVE or indicator across intelligence sources	IMPLEMENTED	`server/ti/compare.ts`, `routes/app.intel.$key.tsx`

#	Workflow	Status	Primary code
W6	Ask an agent: streamed chat with five chartered agents and their allowlisted tools	IMPLEMENTED	`server/agents/*`, `server/chat/*`, `routes/app.chat*.tsx`
W7	Notify: Teams and Slack webhooks for selected events	IMPLEMENTED	`server/notify/notify.ts`

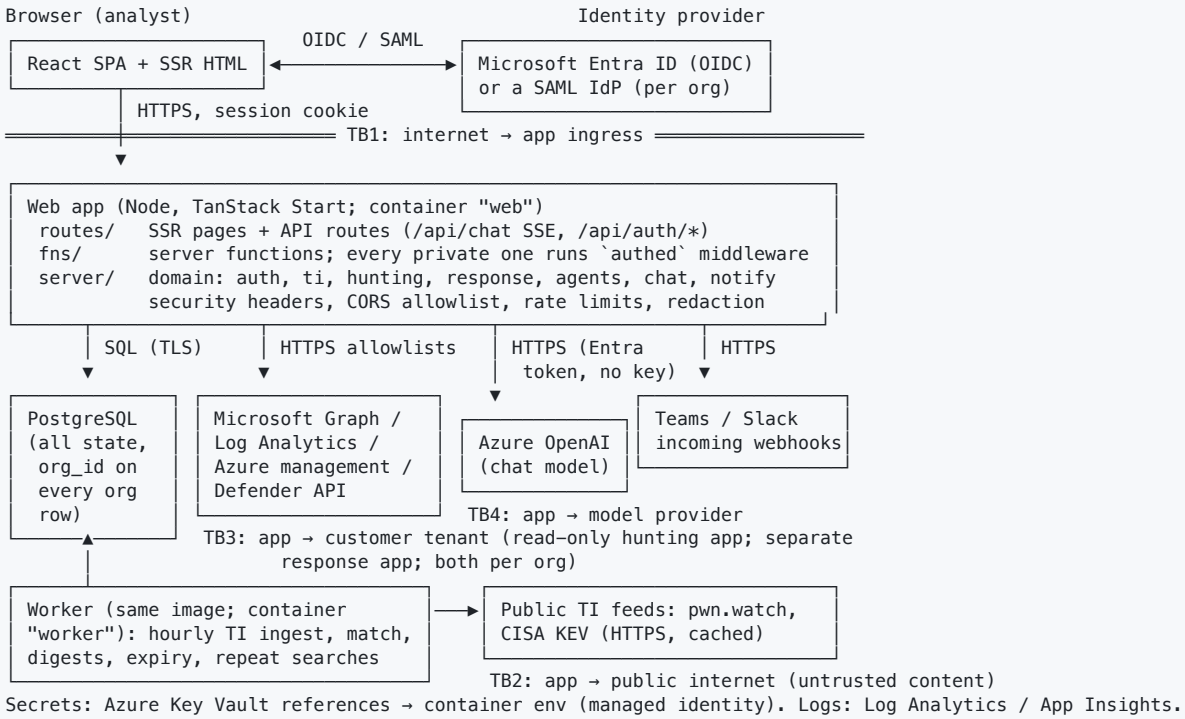
2.3 Out of scope (by design)

Alert queues, detection-rule authoring, entity relationship graphs, risk scoring, ATT&CK mapping, case assignment and storage of full result sets are not part of the product. The interface does not present synthetic metrics or graphs for data the backend does not hold (see `design.md`, section 1).

---

3. Architecture and trust boundaries

3.1 Component diagram



3.2 Trust boundaries

ID	Boundary	What crosses it	Primary controls
TB1	Internet to application	Browser requests, cookies, chat messages, form input	TLS only (ingress `allowInsecure: false`), HSTS, CSP, CORS allowlist, Better Auth sessions, `authed` middleware, zod validation, rate limits
TB2	Application to public internet	Feed JSON from pwn.watch and CISA KEV	Fixed feed URLs from configuration, identifiable User-Agent, timeouts, bounded retries, cache with TTL, schema parsing (zod), rendered as text
TB3	Application to customer tenant	KQL queries and results; response API calls	Separate least-privilege apps per org, request allowlists (host + path + method), KQL safety filter, row and time caps, human approval, dry-run default

ID	Boundary	What crosses it	Primary controls
TB4	Application to model provider	System prompt, conversation, tool results	Entra ID token (managed identity; no stored key), server-chosen model and tools, tool allowlist per agent, no execute tools, output rendered as escaped markdown
TB5	Application to chat-ops endpoints	Notification payloads	HTTPS, default port, no credentials in URL, host allowlist per kind, `redirect: 'error'`, 5 s timeout, payloads carry titles, counts and links only
TB6	Organisation to organisation (logical)	All tenant data in one database	`org_id` on every org-owned row; server code always filters by the session's org; cross-org tests

---

4. Data flows

4.1 Intelligence ingest and matching (W1)

1. The worker (`server/cli/worker.ts`) runs an ingest cycle at start and every hour: `fetchAndIngestPwnWatch` (incidents, changes, tags) and `fetchAndIngestKev`, via `politeFetch` in `server/ti/http.ts` (cache table `ti_fetch_cache`, TTL at least 3600 s by default).
2. Records are parsed with zod schemas; invalid items are skipped and counted, not stored.
3. `server/ti/matcher.ts` matches each org's active watches (keyword and vendor are case-insensitive whole-word, at least 3 characters; CVE and tag are exact) and writes `watch_hits` rows with `org_id`.
4. Five minutes after local midnight in `APP_TIMEZONE`, the worker builds the previous day's digest per org and emails owners when SMTP is configured (`server/ti/digest.ts`). Day bounds are DST-safe, including zones with no local midnight.

4.2 Search approval and execution (W2)

1. An agent tool (`propose_kql`) or an analyst creates a `kql_proposals` row (status `proposed`, expiry set).
2. A person reviews and may edit the KQL, then approves. `approveAndRunKql` (`server/hunting/hunts.ts`) is the only code path that executes KQL and requires a human user id.
3. `assertSafeKql` rejects external data, HTTP, plugin, cross-cluster and management commands. The claim is atomic: only one approver moves `proposed` to `approved`.
4. The hunting client obtains a client-credentials token for the org's read-only app (`server/hunting/broker.ts`), checks the request against `HUNTING_ALLOWLIST`, applies row and timeout caps, and stores a summary (row count, columns, a sample of at most 20 rows) plus an evidence event. In demo mode, labelled fixture rows are returned instead.

4.3 Response (W4)

1. A proposal (`response_proposals`) holds targets, a blast-radius summary and payload. Agents may create proposals only through `propose_response`, on behalf of a member.
2. An approver dry-runs it: executors compute the exact calls and any blockers without sending anything.
3. A different approver approves (self-approval is refused unless `RESPONSE_ALLOW_SELF_APPROVAL=true`).
4. Execution happens only if `RESPONSE_DRY_RUN=false` *and* the executor flag for that action type is `on`. Calls are checked against `RESPONSE_ALLOWLIST`, capped by `RESPONSE_MAX_LIVE_TARGETS`, and recorded per call. Rollback is a new proposal through the same gates.

4.4 Chat (W6)

1. `POST /api/chat` (`server/chat/handler.ts`) authenticates, rate-limits per IP, loads the thread for that user and org, appends the user message and builds history from the database (the client never supplies history).
2. `server/agents/runtime.ts` builds the system prompt from the agent's charter plus org memory and runs the TanStack AI tool loop (at most 6 iterations; 4 for delegated specialists) with only that agent's tools.
3. The model is Azure OpenAI via `server/agents/model.ts`, signed in with `DefaultAzureCredential` (managed identity). The response streams to the browser as Server-Sent Events and is persisted on completion; every turn and tool call writes an `agent_decisions` row.

---

5. Backend components

Backend inventory (paths under `apps/web/src/`)

Component	Responsibility	Contracts and notes	Status
`server/env.ts`	Typed configuration (zod); production refuses to start without `APP_URL`, `DATABASE_URL`, `BETTER_AUTH_SECRET`, `APP_ENCRYPTION_KEY`	Empty values mean unset; invalid values fail fast with the variable name	IMPLEMENTED
`server/auth/*`	Better Auth: email and password (off in Azure), Entra ID OIDC, SAML per org; org placement by tenant id; session lookup	`getAppSession`, `requireApiSession`; Entra users without an email claim fall back to UPN, always unverified	IMPLEMENTED
`fns/*` + `fns/middleware.ts`	Server functions consumed by the UI	Every private function uses `authed`; inputs validated with zod; role checks inside handlers	IMPLEMENTED
`server/ti/*`	Feed ingest, cache, matching, digests, compare, optional Cloudflare Radar enrichment	Global TI tables (not org-scoped); org-scoped watches, hits, digests	IMPLEMENTED (Radar flag-off)
`server/hunting/*`	Hunts, KQL proposals, safety filter, token broker, hunting client, schedules, report export	Request allowlist; row cap at most 1000; timeout at most 120 s; demo fixtures when no connector	IMPLEMENTED
`server/response/*`	Proposals, RBAC, executors (Sentinel playbook, Logic App, Defender isolate), response connector	Dry-run default; two switches for live; per-call results; rollback as proposal	IMPLEMENTED
`server/agents/*`	Charters, tool registry, memory, decision log, runtime, model adapter	`EXECUTION_TOOL_NAMES` must stay empty (tested); Azure OpenAI or OpenAI-compatible	IMPLEMENTED
`server/chat/*`	Threads, messages, feedback, SSE handler	Threads scoped by org and user; history assembled server-side	IMPLEMENTED
`server/notify/notify.ts`	Teams and Slack webhooks	URL validation and encryption at rest; payloads exclude query results and secrets	IMPLEMENTED
`server/lib/*`	AES-256-GCM secret encryption with key rotation, nonce-based CSP and security headers, CORS, rate limiter, JSON logger, redaction	Rate limits are stored in Postgres and shared by every replica with the default `RATE_LIMIT_STORAGE=database`; `memory` keeps per-replica counters	IMPLEMENTED

Component	Responsibility	Contracts and notes	Status
`server/ops/*`	Bootstrap first org and admin, setup checklist, demo fixtures, migration status, control-panel overview queries, daily retention purge	Bootstrap refuses if an org exists or live execution is enabled	IMPLEMENTED
`server/cli/*`	Worker, migrate, bootstrap, ingest, match, digest, expire, compact, fixtures, retention, rotate-secrets, `model:check`	Run as one-off container jobs in Azure	IMPLEMENTED
Observability	JSON logs with request id; Azure Monitor OpenTelemetry when `APPLICATIONINSIGHTS_CONNECTION_STRING` is set; alert rules in Bicep	`apps/web/telemetry.mjs`, `infra/azure/modules/alerts.bicep`	IMPLEMENTED

5.1 HTTP surface

Route	Auth	Notes
`GET /healthz`	None	Process up; no database
`GET /readyz`	None	503 until Postgres is reachable and migrations are current; reports the latest migration tag
`/api/auth/*`	Better Auth	Sign-in, callbacks, sign-out; Better Auth rate limits (sign-in 5 per minute)
`GET /api/me`	Session	401 when signed out
`POST /api/chat`	Session	401 when signed out; per-IP limit `RATE_LIMIT_CHAT_PER_MIN` (default 20); 20,000-character cap; SSE response
Server functions	Session (`authed`)	All application data; role checks per function

---

6. Data model and storage

PostgreSQL holds all state (34 tables; schema in `apps/web/src/server/db/schema/`, migrations in `apps/web/drizzle/`, latest `0004_gaps_retention_sessions_ratelimits`). Tests run the real migrations against in-process PGlite.

Group	Tables	Scope	Sensitive fields and handling
Identity	`user`, `session`, `account`, `verification`, `sso_provider`	Global (Better Auth)	Passwords hashed by Better Auth; OAuth tokens encrypted (`encryptOAuthTokens: true`)
Tenancy	`orgs`, `org_memberships`, `sso_org_bindings`	Org	Entra tenant id binds an org; role per membership
Audit	`audit_events`, `agent_decisions`	Org	Inputs and outputs redacted before storage
Threat intel	`ti_fetch_cache`, `pwn_incidents`, `pwn_changes`, `pwn_tags`, `cve_records`	**Global**	Public data; untrusted content
Watching	`watch_entries`, `watch_hits`, `digests`	Org	None secret
Hunting	`hunt_cases`, `hunt_evidence`, `kql_proposals`, `hunt_schedules`, `org_connectors`	Org	Result samples (at most 20 rows) may hold tenant data; connector client secret AES-256-GCM encrypted or an `env:` reference

Group	Tables	Scope	Sensitive fields and handling
Response	`response_proposals`, `response_events`, `org_response_connectors`	Org	Response secret encrypted; playbook allowlist
Agents and chat	`agent_memories`, `agent_memory_entries`, `chat_threads`, `chat_messages`, `message_feedback`	Org (threads also by user)	Memory entries redacted; chat content may contain tenant data
Notify	`org_notification_channels`	Org	Webhook URL encrypted; only the host is shown

> IMPLEMENTED A daily retention purge (`server/ops/retention.ts`) removes chat older than `RETENTION_CHAT_DAYS` (365), agent decisions older than `RETENTION_DECISIONS_DAYS` (365) and audit events older than `RETENTION_AUDIT_DAYS` (730), and strips stored result rows older than `RETENTION_SAMPLES_DAYS` (180) while keeping row counts. A value of 0 keeps that data forever. Tool outputs in the decision log store result rows as a count only; free text written by the model may still quote values and follows the decisions period. See section 18.

---

7. Frontend: views and shared UI

React 19 with TanStack Start (SSR) and TanStack Router. View state that matters to an investigation lives in the URL, so reload, back and forward, and shared links restore the same view. Routes are in `apps/web/src/routes/` .

View	Route file	Purpose	URL state
Sign-in, auth error	`sign-in.tsx`, `auth-error.tsx`	Entra or SAML sign-in; readable errors without provider payloads	`returnTo` (same-origin paths only)
App shell	`app.tsx`	Grouped navigation, org and role, theme, skip link, focus on navigation	none
Control panel	`app.index.tsx`	Feed status, safety state, decisions waiting, emerging threats, watch hits by day, newly exploited flaws, hunts, agent activity	none
Watches	`app.watches.tsx`	Watch list with per-watch counts; hits table	`q`, `entry`, `day`, `source`
Digests	`app.digests.tsx`	One section per local day; link to that day's hits	none
Hunts list	`app.hunts.tsx`	Dense table, text and status filters, new hunt	`q`, `status`, `fromHit`, `title`
Hunt workspace	`app.hunts.$huntId.tsx`	Context bar, search list, query review, result grid, row inspector, entities, timeline, notes, repeat searches	`q`, `row`, `f`, `s`, `tl`, `ev`
Approvals and detail	`app.approvals*.tsx`	Waiting / decided inbox; targets, blast radius, dry-run calls, decision, timeline	`view`
Compare	`app.intel.$key.tsx`	Field-by-source grid with labelled dates	key in path
Chat	`app.chat*.tsx`	Conversation list by calendar day, streamed markdown, tool steps, stop, copy, rating	thread id in path
Agents and log	`app.agents.tsx`	Agent memory; server-searched, paged decision log	`agent`, `actor`, `q`, `before`
Settings	`app.settings.tsx`	People and roles, hunting and response connectors, notification channels, audit log	anchors

7.1 Shared UI modules

Module	Exports
`components/ui.tsx`	`Mark`, `Wordmark`, `Breadcrumbs`, `PageHead`, `SectionHead`, `Notice`, `Term` (glossary), `TableWrap`, `Count`
`components/signals.tsx`	`SeverityTag`, `Freshness`, `ModeTag` (demo / live / dry run), `StatusTag`, `normaliseSeverity`
`components/results.tsx`	`ResultGrid`, `Inspector`, `FilterChips`; filter and sort URL encoding; `kqlLiteral`, `kqlIdentifier`; `summariseEntities`
`components/ThemeToggle.tsx`	System / light / dark select; pre-paint script in `routes/__root.tsx`
`components/format.ts`	Display time zone, absolute and relative time

---

8. Component and state catalogue

For each interactive component: what it does, keyboard support, accessibility, security notes and the states it must render. "Permission" means the control is absent or disabled for the role *and* the server refuses the action.

8.1 Result grid (`ResultGrid`)

- **Interaction:** sortable columns (sort held in the URL and shared with the inspector); wrap toggle; click or Enter opens a row.
- **Keyboard:** roving tab index; `j`/`k` or arrows move; Enter or Space opens; Escape closes the inspector.
- **Accessibility:** native table, `aria-sort`, selected row marked with `aria-selected` and an inset bar (not colour alone); key hints visible.
- **Security:** cells rendered as text; long values truncated with the full value in the inspector; nothing interpreted as HTML.
- **States:** no sample stored; zero rows returned (stated as a result); filtered to nothing (with clear action); sample smaller than row count (notice: filters apply to the sample only); demo data tag on the search.

8.2 Row inspector (`Inspector`)

- **Interaction:** Fields and Raw JSON views; hide empty fields; per-field Show only, Filter out, Copy, Add to new search; previous and next follow the visible order.
- **Keyboard:** `j`/`k` step rows; Escape closes and returns focus to the row being shown.
- **Accessibility:** side panel on wide screens; at 1180 px and below it is a `role="dialog"` with `aria-modal`, focus moves to its heading, Tab is contained and the rest of the page is `inert`.
- **Security:** "Add to new search" drafts `| where` with `kqlIdentifier` (bracket-quoted names) and `kqlLiteral` (escapes quotes, backslashes, line breaks and control characters). The draft is a proposal: it still needs approval and passes `assertSafeKql`.
- **States:** clipboard unavailable (message); empty values shown as "empty", never blank or zero.

8.3 Query review panel

- **Interaction:** editable KQL with "edited from the proposal" indicator and restore; Approve and run, Reject; expiry shown.
- **Security:** what is approved is exactly what runs and is stored; server enforces expiry, single claim, safety filter and caps.
- **States:** needs review, running, ran, failed (error shown), rejected, expired (with "edit as new search"), demo mode notice.

8.4 Evidence timeline and entities

- Timeline: filter chips (All, Searches, Results, Notes, Schedules) in the URL; entries link to the producing search or to a repeat run's own sample; actors labelled agent, person or system.
- Entities: devices, accounts, files and network values counted within the sample only (stated); selecting one adds a filter.
- States: nothing yet; no entity columns; sample without column metadata (columns inferred).

8.5 Approval decision panel

- **Permission:** analysts see a read-only notice; approvers see Dry run, Approve, Reject and, when approved, Execute. Requesters see a notice that another approver must decide. The server enforces all of these.
- **States:** pending, approved (executor off, dry-run, live), executed, failed or partial, rejected, expired; dry run with blockers ("would be refused"); per-call results matched by position; rollback proposed.

8.6 Chat

- **Interaction:** Enter sends, Shift+Enter new line, Stop reply, Copy, rating; agent selector; suggested questions; draft survives reloads.
- **Accessibility:** transcript is `role="log"`; one polite status per reply ("Intel replied"), not per token; tool steps are a labelled list with pending, done and failed states.
- **Security:** markdown rendered with `react-markdown` and `remark-gfm` without raw HTML; external links open in a new tab with `noopener noreferrer`; no model credentials reach the browser.
- **States:** model not configured (read-only, setup notice), thinking, streaming, stopped, model error with retry, empty conversation.

8.7 Control panel and feed status

- Feed state per required endpoint: live, stale (older than 3 hours, written out), not fetched; the oldest endpoint decides.
- Counts come from separate count queries; lists are capped previews. Waiting items exclude expired and already-approved items.
- Hits-by-day chart: units, window, time zone and zero days are stated; consecutive calendar days are generated without DST errors.

8.8 Forms (settings, watches, notes, new hunt)

- Visible labels; hints and errors linked with `aria-describedby`; errors announced (`role="alert"`); success announced politely; secrets in password inputs with `autocomplete="new-password"`, never re-displayed.
- Removing a watch, deleting a conversation and deleting a notification channel use an inline confirm step; focus moves to Confirm and returns to Delete on Cancel.

8.9 State matrix

State	Rule	Where
Loading	Route pending component with a plain sentence	Every data route
Error	`Notice` with tone "error" and text label; retry where possible	Every data route, forms
Not found	Explains possible causes, including another org	Hunt, approval, conversation, app root
Empty	Says what will appear and how to get there	All lists
Stale	"stale" written next to the age	Feeds, freshness tags

State	Rule	Where
Missing value	"empty" or "not reported"; never zero	Grid, inspector, compare
Unknown severity	Dashed square glyph and "Unknown severity"; never treated as low	Lists, compare
Demo data	Tag on the result and a notice on the hunt; never shown as live	Hunts, hits, digests, approvals
Permission	Explains who can act; server refuses regardless	Approvals, schedules, settings, export
Integration unavailable	Connector not configured: demo mode notice; model not configured: chat read-only	Hunts, chat

8.10 Performance and scale

- Result samples are small (at most 20 rows); filtering and sorting are client-side over the sample.
- Lists are capped server-side (hunts 200, hits 100, or 500 when scoped to a watch or day; decision log paged in 100s).
- IMPLEMENTED With the default `RATE_LIMIT_STORAGE=database`, rate limits are stored in Postgres and shared by every web replica; with `memory` each replica keeps its own counters. Repeat-search claims are atomic, so extra workers are safe.
- GAP No load test has been run. No pagination on the hunts list beyond 200.

---

9. Design tokens and accessibility

Tokens live in `apps/web/src/styles/tokens.css` as CSS custom properties with light and dark themes; components reference semantic roles only. The pattern research and decisions are in `design.md`.

Token group	Examples	Rule
Surfaces	`--bg`, `--surface-1..3`, `--surface-selected`, `--surface-code`	Borders separate; shadows only for overlays
Text	`--text`, `--text-2`, `--text-muted`, `--text-code`	All text pairs at least 4.5:1 in both themes (scripted check)
Accent and focus	`--accent`, `--accent-solid`, `--focus`	One accent; focus 2 px, never animated
Controls	`--border-control`	Input and button boundaries at least 3:1 (WCAG 1.4.11)
Severity	`--sev-critical..unknown`	Always with a shape and a word
Status	`--status-ok/attention/danger/info/neutral` (+ `-bg`)	Outlined tag with text
Provenance	`--mode-demo`	Demo data is visibly distinct
Type and space	`--text-body` 14 px, tables 13 px; 4-point spacing; radii 4/6/8 px	Root size stays at the browser default

9.1 Accessibility commitments

- IMPLEMENTED Automated axe scan of 14 views in both themes, tags `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, `wcag22aa`: 0 violations (`e2e/tests/a11y.spec.ts`).
- IMPLEMENTED Skip link; focus to the page heading after navigation; label-in-name on all controls; 24 px minimum targets; links in text underlined; glossary terms are buttons with dismissible, persistent definitions (WCAG 1.4.13); reduced motion respected; no horizontal scroll at 390 px.

- GAP No manual screen-reader test (NVDA, JAWS, VoiceOver) has been recorded. Automated checks find only part of the issues.

---

10. Roles, tenants and permissions

10.1 Permission matrix

Every row is enforced on the server. "UI" notes how the interface presents it.

Action	Analyst	Approver	Owner	Agent	Enforced in
Read org data (hunts, hits, approvals, log)	Yes	Yes	Yes	Via tools, org-scoped	`authed`; every query filters `org_id`
Add or remove watches; notes; new hunt	Yes	Yes	Yes	No	`fns/watches.ts`, `fns/hunts.ts`
Propose KQL	Yes	Yes	Yes	Yes (`propose_kql`)	`proposeKql`
Approve and run KQL	Yes	Yes	Yes	**No**	`approveAndRunKql` requires a human user id; `KQL_APPROVAL_POLICY` may restrict it (10.2)
Schedule, pause, re-approve repeat search	No	Yes	Yes	No	`createSchedule` (tested)
Propose response action	Yes	Yes	Yes	Lead, Responder	`createProposal`, `AGENT_TOOLS`
Dry run, approve, reject, execute	No	Yes (not own request)	Yes (not own request)	**No**	`assertCanDecide`; lint rule blocks agent imports
Compact agent memory	No	Yes	Yes	No	`fns/agents.ts`
Change roles; connectors; channels; audit log; audit bundle	No	No	Yes	No	`fns/settings.ts`, `fns/hunts.ts`, `fns/notify.ts`, `fns/approvals.ts`
Load demo data	No	No	Yes, only if `ALLOW_DEMO_FIXTURES=true` in production	No	`fns/setup.ts`

> IMPLEMENTED **KQL approval policy** (`KQL_APPROVAL_POLICY`, rules in `apps/web/src/shared/kql-approval.ts`, shared by the server check and the UI notice). `member` (default, decided 08-11-2026): any member may approve and run a read-only search. `approver`: owners and approvers only. `second-person`: an approver other than the person who asked for the search (the proposer, or the chat user whose agent proposed it, stored as `requested_by`), and the query cannot be edited at approval; an edit becomes a new proposal.

10.2 Tenant isolation

- Each org-owned table carries `org_id`; server functions take the org from the session, never from the request body.
- Cross-references are re-checked in the same org (for example, a hunt's origin watch hit resolves only within the org; tested in `test/workspace.test.ts`).
- Entra sign-in maps the token's tenant id to exactly one org; unknown tenants are refused before a user row is created and audited (`sso_rejected_unknown_tenant`).
- Connector secrets use per-org `env:` reference prefixes; a reference outside the org's prefix is refused.
- ASSUMPTION Isolation is enforced in application code. PostgreSQL row-level security is not used; a missing filter in new code would not be caught by the database.

---

11. Threat model

STRIDE-style, per boundary. Likelihood and impact are qualitative for this deployment model (single customer per installation, self-hosted).

ID	Threat	Boundary	Mitigations in code	Residual risk
T1	Session theft or fixation	TB1	Secure, HTTP-only cookies in production; HSTS; Better Auth sessions; CSRF protection via Better Auth origin checks and `trustedOrigins`	Idle timeout 60 minutes and absolute cap 12 hours (configurable)
T2	Cross-tenant data access (IDOR)	TB6	Org from session; every query filters `org_id`; tests for cross-org access	No database-enforced isolation (see 10.2)
T3	Privilege escalation to approve or execute	TB1	Role checks on the server; self-approval off; execution needs two switches; agents have no execute path (lint and tests)	Owner account compromise grants full control; depends on IdP MFA
T4	KQL abuse (exfiltration, management commands, cost)	TB3	Human approval; `assertSafeKql` blocklist; read-only app permissions; row and time caps; request allowlist	Approved KQL can read any table the hunting app can read (documented in `docs/hunting.md`)
T5	SSRF via configurable URLs	TB2, TB5	Feed URLs from server config only; webhook host allowlist, HTTPS, default port, no redirects; hunting and response clients use fixed host and path allowlists	Feed base URLs are trusted configuration
T6	XSS from feed content, results or model output	TB2, TB4	React escaping; markdown without raw HTML; no `dangerouslySetInnerHTML` on untrusted data; CSP with `default-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`	Production CSP uses a per-request script nonce; `style-src` still allows inline styles
T7	Prompt injection through feed text, results or chat	TB4	Agents can only read and propose; every action needs a person; tool allowlists; tool arguments validated by zod; decisions logged	A model can still produce misleading text or proposals; reviewers must read them
T8	Secret disclosure in logs, memory or UI	All	Redaction of tokens, JWTs, keys and secret-named fields in logs, audit, decisions and memory; secrets never returned to the browser; Key Vault references in Azure	Pattern-based redaction can miss novel formats

ID	Threat	Boundary	Mitigations in code	Residual risk
T9	Supply-chain compromise	Build	Pinned actions by SHA; frozen lockfile; SBOM (SPDX); Trivy gate on fixable critical CVEs; non-root container	Deploys run images by digest; a promoted release is verified (`gh attestation verify`, source ref) before deploy. Images built inside the same deploy run are trusted from that run
T10	Denial of service	TB1, TB4	Per-IP chat limit; Better Auth limits; message size cap; agent timeout; model iterations capped	Limits are shared through Postgres; the chat limiter fails open if the database errors; no WAF
T11	Unauthorised live containment	TB3	Dry-run default; executor flags; target cap; allowlisted playbooks; per-call audit; rollback as proposal	Requires operational change control to stay effective
T12	Account takeover via email claim	TB1	Entra UPN fallback always unverified; Microsoft provider not trusted for implicit linking; org placement by tenant id	Local accounts can exist if email sign-up is enabled (off in Azure)

---

12. Security controls in depth

12.1 Authentication, session, SSO and MFA

- IMPLEMENTED Entra ID OIDC (`openid profile email` only), single tenant recommended; SAML per org (`@better-auth/sso`); email and password disabled in Azure (`AUTH_EMAIL_PASSWORD=false`).
- ASSUMPTION MFA and phishing-resistant authentication are enforced by the identity provider (Conditional Access). The application does not implement its own second factor. Align the IdP policy with NIST SP 800-63B-4 at AAL2 or above, preferring phishing-resistant methods for owners and approvers.
- IMPLEMENTED Sessions: idle timeout `SESSION_IDLE_MINUTES` (60) slides with activity; absolute cap `SESSION_MAX_AGE_HOURS` (12) since sign-in is enforced on every app request and in front of the auth endpoints; over-age sessions are revoked and audited (`session_expired_absolute`).
- IMPLEMENTED Bootstrap: the first admin email is bound to the org; the first matching sign-in becomes owner.

12.2 Least privilege

- Three separate Entra apps: SSO (sign-in only), hunting (`ThreatHunting.Read.All`, Log Analytics Reader, Sentinel Reader), response (`Machine.Isolate`, Sentinel Responder, Logic App Contributor on allowlisted playbooks). See `docs/runbooks/entra-setup.md`.
- Deployment identity: GitHub OIDC federation to a Contributor role on the resource group only; no long-lived deploy secret.
- Runtime identity: user-assigned managed identity with Key Vault Secrets User and Cognitive Services OpenAI User only.
- Azure OpenAI resource with local (key) authentication disabled in the lab.

12.3 Secrets and key management

- Azure Key Vault holds `database-url`, `better-auth-secret`, `app-encryption-key`, `entra-client-secret`; Container Apps resolve them with the managed identity.
- Connector and webhook secrets are encrypted with AES-256-GCM under `APP_ENCRYPTION_KEY` (`server/lib/crypto.ts`, format `v1.iv.tag.ciphertext`).

- IMPLEMENTED Key rotation: decryption also accepts `APP_ENCRYPTION_KEY_PREVIOUS`; `pnpm rotate-secrets` re-encrypts every stored secret under the current key in one transaction and reports counts only (`docs/runbooks/key-rotation.md`).

12.4 Transport and storage encryption

- Ingress is HTTPS only; HSTS one year with subdomains. Database connections use `sslmode=require`.
- ASSUMPTION Storage encryption at rest is provided by the platform (Azure Database for PostgreSQL and Key Vault defaults). No customer-managed keys are configured.

12.5 Input, query and injection protection

- All server-function inputs are validated with zod; SQL is built with Drizzle (parameterised); `ILIKE` search terms escape `%`, `_` and backslash.
- KQL: human approval, `assertSafeKql` blocklist (external data, HTTP, Python and R plugins, SQL request, management commands, cross-cluster), length limit 10,000.
- Generated KQL drafts escape literals and quote identifiers.

12.6 Untrusted intelligence ingestion

- Fixed URLs, timeouts, bounded retries with back-off on 429 and 503, identifiable User-Agent, cache TTL at least one hour.
- Schema-validated; invalid items skipped and counted; content stored and rendered as text; links open with `noopener noreferrer`.
- IMPLEMENTED Feed responses larger than `TI_MAX_RESPONSE_BYTES` (25 MB) are refused, by Content-Length and while streaming, and nothing is cached.

12.7 Safe rendering of links, files and graphs

- No graph rendering exists. Exports (hunt report Markdown, audit bundle JSON) are generated server-side and downloaded as files; the report redacts secrets and caps size (`REPORT_MAX_CHARS`).

12.8 Audit, redaction, privacy and retention

- `audit_events`: sign-ins, rejections, role changes, connector and channel changes, response events, bootstrap, exports. `agent_decisions`: every agent turn and tool call, redacted.
- Logs are JSON with request ids; authorisation headers, cookies, tokens and secrets are redacted.
- Bootstrap audit stores no full email; people are shown by email only to members of the same org.
- IMPLEMENTED Retention periods (section 6). GAP Deleting one person's data on request is a manual database task; there is no self-service export or erasure.

12.9 Supply chain and CI security

- GitHub-hosted runners; actions pinned to commit SHAs; `pnpm install --frozen-lockfile`; lint, typecheck, tests, build and migration drift check on every PR.
- Image job: build, SBOM (Anchore, SPDX), Trivy gate failing on fixable critical CVEs; image pushed to GHCR by immutable SHA tag.
- Container runs as the `node` user with a health check.
- IMPLEMENTED Dependabot (Actions, npm, Docker, weekly); CodeQL for TypeScript and workflow files on every PR, push and weekly; signed build provenance and SBOM attestations bound to the pushed digest; release tags are published only after both attestations succeed (`docs/releasing.md`).

13. AI agent safety

Control	Implementation	Reference
No excessive agency	Tools can read and propose only; `EXECUTION_TOOL_NAMES` empty and tested; lint blocks agents from importing executors or decision functions	OWASP LLM06 Excessive Agency
Per-agent allowlist	`AGENT_TOOLS` in `server/agents/tools.ts`	OWASP LLM06
Authorisation outside the model	Every tool re-checks org from server context; approvals by people only	OWASP LLM06
Prompt injection tolerance	Feed text, results and chat cannot trigger execution; outputs rendered as escaped markdown	OWASP LLM01, LLM05
Sensitive data	Memory and decisions redacted; model credentials never reach the browser	OWASP LLM02
Unbounded consumption	Iteration caps, timeout, per-IP limit, message size cap	OWASP LLM10
Model access	Azure OpenAI via managed identity; deployment chosen server-side	ADR 0004

14. Configuration and dependencies

14.1 Key configuration

Variable	Default	Effect
`APP_URL`, `DATABASE_URL`, `BETTER_AUTH_SECRET`, `APP_ENCRYPTION_KEY`	none in production	Required; start-up fails without them
`AUTH_EMAIL_PASSWORD`	`true` locally, `false` in Bicep	Local accounts
`ENTRA_CLIENT_ID`, `ENTRA_TENANT_ID`, `ENTRA_CLIENT_SECRET`	unset	Enables Entra sign-in
`HUNTING_DEMO_MODE`, `HUNTING_MAX_ROWS`, `HUNTING_TIMEOUT_MS`	`true`, 100, 60 s	Demo fixtures when no connector; caps (hard limits 1000 rows, 120 s)
`RESPONSE_DRY_RUN`	unset = on	Live execution needs an explicit `false`
`RESPONSE_EXECUTOR_SENTINEL|LOGICAPP|GRAPH`	`off`	`off`, `dry-run` or `on`
`RESPONSE_ALLOW_SELF_APPROVAL`, `RESPONSE_MAX_LIVE_TARGETS`, `RESPONSE_PROPOSAL_TTL_HOURS`	`false`, 10, 24	Decision rules and caps
`MODEL_PROVIDER`, `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_DEPLOYMENT`, `AZURE_CLIENT_ID`	`off`	Chat model; keyless via managed identity
`AGENT_TIMEOUT_MS`, `RATE_LIMIT_CHAT_PER_MIN`, `RATE_LIMIT_STORAGE`	90 s, 20, `database`	Chat limits; `database` shares counters across replicas, `memory` keeps them per replica
`KQL_APPROVAL_POLICY`	`member`	`member`, `approver` or `second-person`
`SESSION_IDLE_MINUTES`, `SESSION_MAX_AGE_HOURS`	60, 12	Idle timeout and absolute session cap
`RETENTION_CHAT_DAYS`, `RETENTION_DECISIONS_DAYS`, `RETENTION_SAMPLES_DAYS`, `RETENTION_AUDIT_DAYS`	365, 365, 180, 730	Daily purge; 0 keeps forever

Variable	Default	Effect
`TI_MAX_RESPONSE_BYTES`	25 MB	Feed download cap
`APP_ENCRYPTION_KEY_PREVIOUS`	unset	Old keys accepted for decryption during rotation
`APP_TIMEZONE`	`Australia/Sydney`	Display and digest day
`ALLOW_DEMO_FIXTURES`	unset	Allows demo data in production (labs only)

Full list: `.env.example` and `docs/configuration.md`.

14.2 Version assumptions

Dependency	Version at commit
Node.js	24 (`.nvmrc`); image `node:24-bookworm-slim`
pnpm	10.11.0
React, React DOM	19.3.1
TanStack Start / Router	1.169.4 / 1.170.46
TanStack AI (+ OpenAI adapter)	1.2.0 (1.1.1)
Better Auth (+ SSO)	1.7.18
Drizzle ORM, postgres.js	0.45.4, 3.4.9
@azure/identity	4.14.1
zod	4.6.7
PostgreSQL	17 (Compose and Azure Flexible Server)

---

15. Deployment and operations

15.1 Topology (Azure)

Bicep in `infra/azure/`: Container Apps environment with `web` (1 to 3 replicas, probes on `/healthz` and `/readyz`), `worker` (exactly 1) and a manual migration job; PostgreSQL Flexible Server (Burstable B1ms, 7-day backups); Key Vault (RBAC); Log Analytics and Application Insights; alert rules; user-assigned managed identity; optional Azure OpenAI role assignment. Docker Compose files support local and self-hosted use.

> IMPLEMENTED `privateNetworking=true` (Bicep) puts the Container Apps environment and PostgreSQL in a VNet with a private DNS zone and public database access disabled. ASSUMPTION Not yet deployed anywhere; an existing environment cannot be switched in place. The lab still uses `postgresPublicAccess=true` ("Allow Azure services" firewall rule). Production should use private networking (VNet integration or a private endpoint) as described in `docs/runbooks/azure-deploy.md`.

15.2 Release and rollback

1. Deploy workflow (manual dispatch on `main`): CI, image build with SBOM and CVE gate, then the `production` environment with required reviewers.
2. Records the current revision, runs the migration job, rolls out web and worker, waits for the new revision, runs the smoke test.

3. Rollback: re-point to the previous revision or image tag (`docs/runbooks/rollback.md`). Migrations are forward-only; schema changes must stay backward compatible for one release.

15.3 Backup and restore

Point-in-time restore of the database (7-day retention). A restore drill and recorded RPO/RTO are part of the go-live checklist (`docs/go-live-checklist.md`, `docs/runbooks/backup-restore.md`). PLANNED The drill has not been recorded for the lab.

15.4 Operational runbook (summary)

Task	How
Health	`scripts/smoke.sh <url> <migration-tag>`; `/readyz`
Model check	Run `src/server/cli/model-check.ts` as a one-off job with the app's environment
Bootstrap	`docs/runbooks/bootstrap.md`
Custom domain and TLS	`docs/runbooks/custom-domain.md`
Logs	Log Analytics: `ContainerAppConsoleLogs_CL`; alert on 5xx, probe failures and failed hunts or executions
Live execution	Change ticket, then `RESPONSE_DRY_RUN=false` and the executor flag; review `docs/response.md`

---

16. Testing and acceptance criteria

16.1 Test inventory at the commit

- Unit and integration (Vitest, real migrations on PGlite): 197 tests in 21 files; 195 pass and 2 are skipped (run locally on 8 November 2026); covers auth, RBAC, tenant isolation, hunting safety, schedules, response gates, notifications, TI, digests and time zones, agents and chat with a mock model.
- End to end (Playwright, local, not in PR CI): golden path, hunt workspace (keyboard, filters, history, theme, 390 px and 1000 px dialogs), chat streaming (deterministic mock), axe scan.
- End to end on a local production build of this branch (8 November 2026): all 6 Playwright specs pass with the nonce CSP active and no CSP violations, including the axe WCAG 2.2 AA scan.
- Live lab check (8 November 2026): smoke test, in-container Azure OpenAI call via managed identity, Entra sign-in, hunt run, analyst read-only approval, streamed chat with a tool call.

16.2 Acceptance criteria

AC-01 An analyst cannot dry-run, approve, reject or execute a response proposal; the server returns an error even if the request is sent directly.

AC-02 A requester cannot approve their own proposal unless `RESPONSE_ALLOW_SELF_APPROVAL=true`.

AC-03 No agent tool can execute KQL or a response action; `EXECUTION_TOOL_NAMES` is empty.

AC-04 With default configuration, no response call leaves the application (dry-run on, executors off).

AC-05 A user in org A cannot read or change any org B record through any server function or API route.

AC-06 A KQL proposal containing a blocked operator is rejected before execution.

AC-07 Webhook URLs outside the allowed hosts, with credentials, a non-default port or HTTP are rejected.

AC-08 No secret value appears in logs, audit events, decision rows, memory entries or API responses.

AC-09 Feed, result and model text is never interpreted as HTML in the browser.

AC-10 Every view passes the axe scan for WCAG 2.2 AA in light and dark themes with 0 violations.

AC-11 All interactive features are usable with a keyboard alone; focus is visible and returns sensibly after dialogs close.

AC-12 Reload and browser back restore the hunt workspace view (search, row, filters, sort, timeline).

AC-13 Demo data is labelled at every place it appears; it is never presented as live.

AC-14 `/readyz` returns 503 until the database is reachable and migrations are current.

AC-15 The image build fails on a fixable critical vulnerability.

---

17. Standards mapping

This is guidance alignment, not a compliance or certification claim.

Standard	Version used	Where it applies here
OWASP Top 10	2025	A01 access control and SSRF (sections 10, 12); A02 misconfiguration (14, 15); A03 supply chain (12.9); A04 crypto (12.3, 12.4); A05 injection (12.5); A07 authentication (12.1); A09 logging and alerting (12.8, 15); A10 exceptional conditions (8.9 states, fail-closed executors)
OWASP ASVS	5.0.0 (May 2025)	Use as the verification checklist; target Level 2 for this application. Reference requirements with version-tagged ids (for example `v5.0.0-x.y.z`). No formal assessment has been done.
OWASP Top 10 for LLM Applications	2025	LLM01, LLM02, LLM05, LLM06, LLM10 (section 13)
NIST SP 800-63B-4	Final (2025)	IdP policy: AAL2 or above; phishing-resistant authenticators preferred for owners and approvers (section 12.1)
NIST SP 800-218 (SSDF)	1.1 final; 1.2 draft (December 2025)	Secure development practices: reviewed changes, pinned dependencies, SBOM, vulnerability gate, staged release and rollback (sections 12.9, 15.2)
NIST SP 800-207	Final (August 2020)	Per-request authorisation, least privilege, no implicit trust between components (sections 3, 10, 12.2)
CIS Docker Benchmark	1.9.0	Container image and runtime hardening (non-root user, minimal base, health check)
CIS Microsoft Azure Foundations Benchmark	current CIS release (6.0.0 or later)	Subscription baseline: Key Vault, PostgreSQL networking, logging, identity ASSUMPTION not assessed
WCAG	2.2 (W3C Recommendation, 12 December 2024 edition)	Section 9 and acceptance criteria AC-10, AC-11

---

18. Gaps, open decisions and backlog

Item	Type	Detail	Suggested phase
Data retention	IMPLEMENTED	Daily purge; defaults 365 / 365 / 180 / 730 days (decided 08-11-2026)	Done
Database-level isolation	GAP	Consider PostgreSQL row-level security as defence in depth for `org_id`	2
CSP script policy	IMPLEMENTED	Per-request nonce; verified in a production build with no violations	Done
Feed size limit	IMPLEMENTED	`TI_MAX_RESPONSE_BYTES`	Done

Item	Type	Detail	Suggested phase
Key rotation	IMPLEMENTED	`APP_ENCRYPTION_KEY_PREVIOUS` plus `pnpm rotate-secrets`	Done
Supply chain	IMPLEMENTED	Dependabot, CodeQL, provenance and SBOM attestations; release promotion verifies attestations and deploys the verified digest	Done
Private networking	PARTIAL	Opt-in `privateNetworking` Bicep mode added and linted, not yet deployed; the lab keeps public access. Key Vault private endpoint and NSGs are follow-ups	Before production
Screen-reader testing	GAP	Manual NVDA and VoiceOver passes	1
Load testing	GAP	No measured capacity. Rate limits are shared across replicas when stored in the database (the default)	2
KQL approval role	IMPLEMENTED	Decided 08-11-2026: configurable `KQL_APPROVAL_POLICY`, default `member`	Done
Session lifetime	IMPLEMENTED	Decided 08-11-2026: 1 hour idle, 12 hours absolute, configurable	Done
Repeat-run samples in report	IMPLEMENTED	Issue #99: five redacted rows per run	Done
Digest contact address	IMPLEMENTED	Issue #102: confirmed by an emailed link (needs SMTP)	Done
Live hunting against a real tenant	PLANNED	Lab has no Defender or Sentinel connector; demo mode is on	1
Restore drill	PLANNED	Record RPO and RTO for the lab	Before production
Unused dependency	IMPLEMENTED	`@tanstack/ai-anthropic` removed	Done
Channel delete confirm	IMPLEMENTED	Inline confirm step	Done
Shared rate limits	IMPLEMENTED	Postgres-backed for sign-in and chat	Done
Retention query indexes	IMPLEMENTED	`created_at` indexes on chat messages, evidence, decisions and audit events (migration 0004)	Done
Data subject requests	GAP	No self-service export or erasure for one person's data	2
Custom domain	PLANNED	Lab hostname change pending DNS records	1

19. Traceability and references

19.1 Repository

Topic	Paths
Design decisions and pattern matrix	`design.md`
Architecture decisions	`docs/adr/0002-squad-sdk-embed.md`, `docs/adr/0004-chat-model-azure-openai.md`

Topic	Paths
Authentication and tenants	`docs/auth.md`, `apps/web/src/server/auth/`, `apps/web/src/server/orgs.ts`
Hunting	`docs/hunting.md`, `apps/web/src/server/hunting/`, `apps/web/src/shared/kql-approval.ts`
Response	`docs/response.md`, `apps/web/src/server/response/`
Agents and chat	`docs/agents.md`, `docs/chat-model.md`, `.squad/`, `apps/web/src/server/agents/`
Threat intelligence	`docs/watchlists.md`, `docs/ti/`, `apps/web/src/server/ti/`
Security headers and limits	`apps/web/src/server/lib/http-security.ts`, `redact.ts`, `crypto.ts`
Infrastructure	`infra/azure/`, `Dockerfile`, `docker-compose*.yml`
CI and release	`.github/workflows/`, `docs/releasing.md`
Retention and keys	`apps/web/src/server/ops/retention.ts`, `apps/web/src/server/cli/rotate-secrets.ts`, `docs/runbooks/data-retention.md`, `docs/runbooks/key-rotation.md`
Runbooks	`docs/runbooks/`, `docs/go-live-checklist.md`, `docs/day-1-operator-playbook.md`
Tests	`apps/web/test/`, `e2e/tests/`

19.2 External references

- OWASP Top 10:2025. https://owasp.org/Top10/2025/
- OWASP Application Security Verification Standard 5.0.0. https://asvs.owasp.org/; https://github.com/OWASP/ASVS
- OWASP Top 10 for LLM Applications 2025 (GenAI Security Project). https://genai.owasp.org/
- NIST SP 800-63-4 Digital Identity Guidelines, including SP 800-63B-4. https://pages.nist.gov/800-63-4/sp800-63b/authenticators/
- NIST SP 800-218 Secure Software Development Framework (1.1 final; 1.2 draft). https://csrc.nist.gov/projects/ssdf/publications
- NIST SP 800-207 Zero Trust Architecture. https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf
- CIS Docker Benchmark. https://www.cisecurity.org/benchmark/docker
- CIS Benchmarks list (Azure Foundations). https://www.cisecurity.org/cis-benchmarks
- W3C Web Content Accessibility Guidelines 2.2. https://www.w3.org/TR/WCAG22/

---

20. Rendering this document

The HTML file is the source; the PDF is generated from it with headless Chrome so text stays selectable and links stay live.

```
bash docs/design/render.sh   # writes docs/design/analyst-workspace-design.pdf
```

The script uses Google Chrome or Chromium in headless mode with `--print-to-pdf`, A4 page size from the document's `@page` rule, no browser headers or footers, and outline generation for bookmarks where the browser supports it. Review the PDF for clipped tables after any edit.